

SQL DB-RAG Chatbot - Project Specification

🎯 Project Objective

Build an **AI-powered assistant** that can answer natural language questions based on structured data provided as CSV files.

Core Capabilities

Your system should:

- ✓ **Interpret** user questions in natural language
 - ✓ **Identify** the relevant dataset(s) from available CSV files
 - ✓ **Generate** and run appropriate SQL (or Pandas) queries
 - ✓ **Return** clear, human-readable answers
-

📊 Provided Datasets

You will work with **three CSV files** containing telecommunications data:

1. `sample_bo_tbl_large.csv` (Daily Usage & Transactions)

Column	Description
country	3-letter country code (AAA, BBB, CCC, etc.)
date	Date of record
total_mins	Total minutes usage
international_mins	International minutes usage
sms	SMS count
total_data_usage	Data usage amount
payg_amount	Pay-as-you-go amount

2. `sample_sub_details_large.csv` (Monthly Subscriber Stats)

Column	Description
country	3-letter country code
channel	Sales/service channel
date	Date of record
subs	Active subscribers count
netadds	Net additions
churn	Churn count

3. `sample_revenue_large.csv` (Monthly Revenue)

Column	Description
country	3-letter country code
channel	Sales/service channel
date	Date of record
revenue	Gross revenue
net_revenue	Net revenue

Example Queries Your Assistant Should Handle

Basic Queries

- "What was the churn in AAA last month?"
- "Show total SMS sent by BBB this year."
- "What was the average pay-as-you-go amount in CCC last quarter?"

Intermediate Queries

- "Compare net revenue across channels for DDD over the last 6 months."
- "List countries with highest data usage in Q2."
- "Show monthly net additions for AAA over the past year."

Complex Queries

- "Which country had the highest churn-to-subscriber ratio in Q1?"
 - "What's the correlation between data usage and pay-as-you-go top-ups by country?"
 - "Compare average net revenue of Online vs Retail channels in the last 3 months."
 - "Which countries had both above-average revenue and below-average churn last month?"
-

Technical Requirements

1 LLM Integration

- **Use:** Public LLM (OpenAI, Claude, Mistral) or HuggingFace models
- **The model should:**
 - Select relevant dataset(s)
 - Formulate the query logic (SQL/Pandas)
 - Optionally provide reasoning

2 Query Engine

- **Options:** Pandas, DuckDB, SQLite, or equivalent
- **Requirement:** Generated query should execute successfully on the data

3 Interface

- **Options:** CLI, web app, or Jupyter notebook
- **Requirement:** Accept user queries in natural language

🏆 Bonus Features (Optional)

- ✨ Include reasoning/explanation in responses
 - 🔍 Show generated query to the user
 - 🛡️ Add fallback handling for ambiguous or incomplete queries
-

Deliverables

Code Repository Structure

```
genai_structured_qa/
├── data/                                # CSV input files
│   ├── sample_bo_tbl_large.csv
│   ├── sample_sub_details_large.csv
│   └── sample_revenue_large.csv
├── src/                                 # Source code
│   ├── llm_interface/                  # LLM integration
│   │   └── query_generator.py
│   ├── query_engine/                  # Query execution
│   │   └── pandas_engine.py
│   ├── data_manager/                  # Data handling
│   │   └── loader.py
│   └── core/                           # Core logic
│       └── chatbot.py
├── prompts/                            # LLM prompts
│   └── query_prompt.txt
├── cli/                                # Command line interface
│   └── cli_app.py
├── main.py                             # Main entry point
├── requirements.txt                     # Dependencies
└── README.md                           # Documentation
```

Required Documentation (`README.md`)

- 🛠 Setup and run instructions
- 💡 Example queries and outputs
- 📄 Description of your prompting strategy

Evaluation Criteria

Criterion	Weight	Description
Accuracy of Answers	30%	How correctly the system answers user queries
Code Quality & Modularity	25%	Clean, maintainable, well-structured code
LLM Prompting Clarity	25%	Effectiveness of prompts in generating accurate queries
Solution Usability	20%	Ease of use and user experience

Success Metrics

Minimum Viable Product (MVP)

- ☐ Handles all basic queries accurately
- ☐ Clean, modular codebase
- ☐ Working CLI interface
- ☐ Clear setup instructions

Advanced Features

- ☐ Handles complex multi-dataset queries
- ☐ Provides query explanations
- ☐ Robust error handling
- ☐ Web interface or advanced CLI

Excellence Indicators

- ☐ Handles ambiguous queries intelligently
 - ☐ Provides data visualization
 - ☐ Comprehensive testing suite
 - ☐ Production-ready deployment options
-

Getting Started Checklist

1. **Environment Setup**
 - ☐ Create virtual environment
 - ☐ Install required dependencies
 - ☐ Set up LLM API keys
2. **Data Preparation**
 - ☐ Download/prepare CSV files
 - ☐ Validate data schemas
 - ☐ Create data loading utilities
3. **Core Development**
 - ☐ Implement LLM interface
 - ☐ Build query engine
 - ☐ Create main chatbot logic
4. **Interface Development**
 - ☐ Build CLI application
 - ☐ Add query validation
 - ☐ Implement error handling
5. **Testing & Documentation**
 - ☐ Test with example queries
 - ☐ Write comprehensive README

- [] Document prompting strategy
-

💡 Pro Tips for Success

1. **Start Simple:** Begin with basic queries and gradually add complexity
2. **Prompt Engineering:** Spend time crafting effective LLM prompts
3. **Error Handling:** Plan for edge cases and malformed queries
4. **Data Validation:** Ensure CSV data quality and consistency
5. **User Experience:** Make the interface intuitive and helpful
6. **Testing:** Create a comprehensive test suite with example queries